

LECTURE 4

Socket Programming: TCP & UDP

Protocols, Framing & Advanced Patterns

Why Do We Need Sockets?

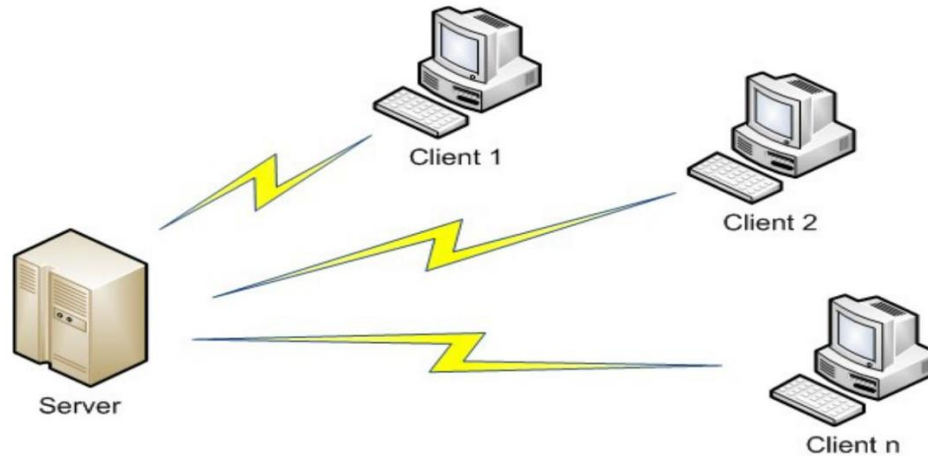
- **Think of a postal address:**
 - A letter needs: a destination address, a room/person name (port), an envelope (packet)
- Programs on different computers also need an addressing system to find each other
- A socket is the programming abstraction that enables this — it gives your program a numbered endpoint (IP + port) that other programs can reach
- **Without sockets:**
 - programs run in isolation — no data exchange, no distributed systems, no web, no databases
- Every network application you use — browser, email client, video call, game — is built on sockets
- Java's `java.net` package wraps OS sockets into clean, object-oriented classes

What Is a Socket?

- A socket is an endpoint for two-way communication between two programs running on a network
- **Uniquely identified by:**
 - IP address — which machine
 - Port number — which program on that machine
 - Protocol — TCP (reliable) or UDP (unreliable, fast)
- **Two types you will use:**
 - `java.net.Socket` — the client-side socket; you connect it to a server
 - `java.net.ServerSocket` — the server-side socket; it listens for incoming connections
- When a client connects to a server, TWO sockets are involved: one at each end
- The OS assigns a random ephemeral port to the client; the server uses its well-known port (e.g. 8080)
- Analogy: IP address = building address, port = office number, socket = phone on the desk

A Typical Client-Server Connection

A Typical Client-Server Connection



Multiple clients connect to one server — each over its own dedicated socket

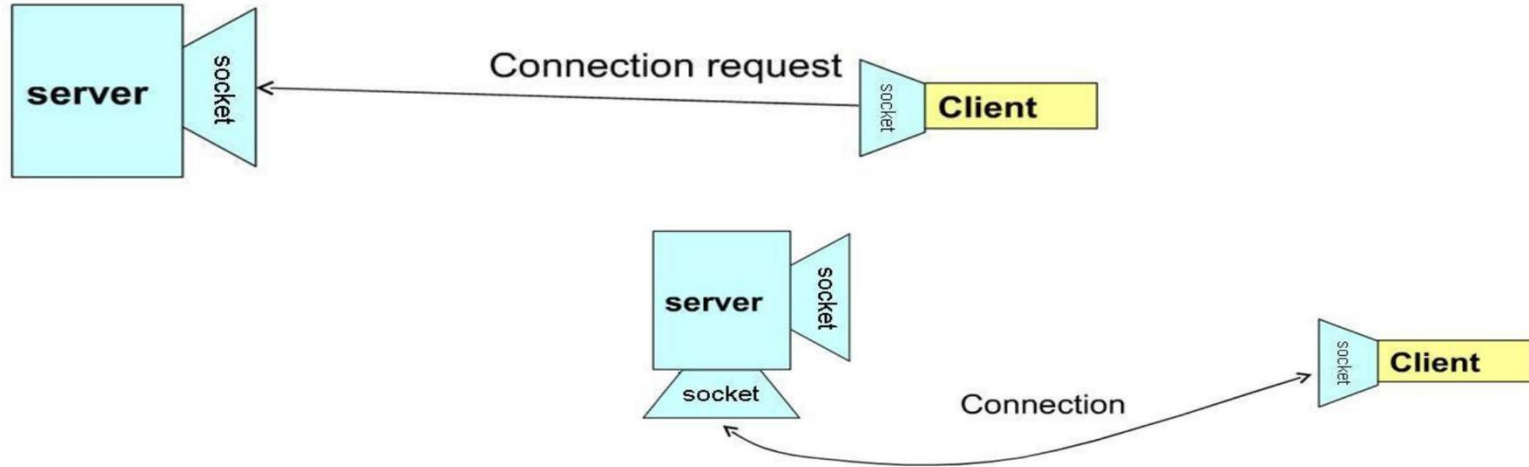
From the Client's Perspective

- **Step 1 — Create socket:**
 - `new Socket(serverIP, port)` — OS performs the TCP 3-way handshake automatically
- **Step 2 — Get streams:**
 - `socket.getInputStream()` and `socket.getOutputStream()` — your read/write channels
- **Step 3 — Communicate:**
 - wrap streams (`BufferedReader`, `PrintWriter`) then read/write data
- **Step 4 — Close:**
 - `socket.close()` — sends TCP FIN, tells the server you are done
- The client actively initiates; the server passively waits
- **Key insight:**
 - after `new Socket()` returns, the connection is already established — the handshake happened inside that constructor call

From the Server's Perspective

- **Step 1 — Create ServerSocket:**
 - `new ServerSocket(port)` — binds to the port and starts listening; nothing connects yet
- **Step 2 — Call `accept()`:**
 - `ServerSocket.accept()` — this BLOCKS the calling thread until a client connects
- **Step 3 — Receive a new Socket:**
 - `accept()` returns a brand-new Socket object representing THAT specific client connection
- **Step 4 — Spawn handler:**
 - pass the new Socket to a thread so the server can loop back to `accept()` for the next client
- **Step 5 — Communicate + close:**
 - handler reads/writes via the client socket, then closes it when done
- Critical: `ServerSocket` is NOT the communication socket — it only produces communication sockets via `accept()`

Connections Illustrated



Connections Illustrated

When a client connects, the server creates a NEW socket for that client — the original listening socket keeps waiting

Key Socket Definitions

Terms You Must Know

- Socket — endpoint combining IP + port + protocol
- Port — 16-bit number (0–65535) identifying an application
- Well-known ports — 0–1023 (HTTP=80, HTTPS=443, SSH=22)
- Ephemeral port — OS-assigned random port for client side
- Binding — associating a socket with a local address/port
- Listening — server waiting for connection requests

Connection Lifecycle

- SYN — client requests connection
- SYN-ACK — server acknowledges
- ACK — client confirms: connection open
- PSH/ACK — data transfer
- FIN/ACK — orderly connection close (4 steps)
- RST — abrupt connection reset (error or timeout)

Socket Streams: Entry & Exit Points

Socket Streams

- A socket serves as **both an entry point to a computer from the network and an exit point** from the computer into the network.
- The function it performs depends on the data streams defined for it.
- A socket can use both an input data stream and an output data stream, or just one stream.
- If only one stream is used, it's considered a read-only or write-only connection.

java.net.Socket — Key Methods

- **Constructors:**
 - `new Socket(String host, int port)` — connects immediately; blocks until handshake completes
 - `new Socket()` then `socket.connect(addr, timeout)` — separate creation & connect with timeout
- **Stream access:**
 - `getInputStream()` — read bytes from the remote end
 - `getOutputStream()` — write bytes to the remote end
- **Socket options:**
 - `setSoTimeout(ms)` — read timeout; `setSendBufferSize(n)`; `setTcpNoDelay(bool)`
- **Half-close:**
 - `shutdownInput()` / `shutdownOutput()` — close one direction while keeping the other open
- **Lifecycle:**
 - `isConnected()`, `isClosed()`, `isBound()` — query state; `close()` — releases OS resources
- Always close sockets in a finally block or try-with-resources to avoid file descriptor leaks

java.net.ServerSocket — Key Methods

- **Constructor:**
 - `new ServerSocket(port)` — binds to port and starts listening immediately
 - `new ServerSocket(port, backlog)` — backlog = max pending connection queue size (default 50)
- **Core method:**
 - `accept()` — BLOCKS until a client connects; returns a connected Socket for that client
 - You MUST loop around `accept()` to handle multiple clients sequentially or in threads
- **Options:**
 - `setReuseAddress(true)` — call BEFORE bind; avoids 'address already in use' on restart
 - `setSoTimeout(ms)` — `accept()` throws `SocketTimeoutException` after ms (use for graceful shutdown)
- **Shutdown:**
 - `close()` — stops accepting new connections; existing connections continue until their own `close()`
- **Pattern:** `while (!shutdown) { Socket client = ss.accept(); new Thread(new Handler(client)).start(); }`

Today's Agenda

- 1 Socket API In Depth
- 2 Designing TCP Protocols: Framing & Streams
- 3 Multi-Client Server Patterns
- 4 UDP Programming Deep Dive
- 5 Socket Robustness & Diagnostics
- 6 Real-World Patterns & Summary

PART 1

Socket API In Depth

Stream wrapping, socket options & the framing challenge

Socket Stream Wrapping Patterns

- Raw streams (getInputStream/getOutputStream): byte-level, no formatting
- Text protocol: wrap with BufferedReader + PrintWriter — line-delimited communication
- Binary protocol: wrap with DataInputStream + DataOutputStream — typed primitive I/O
- Object protocol: ObjectInputStream + ObjectOutputStream — Java serialization
- Always add BufferedInputStream/BufferedOutputStream — reduces OS system calls dramatically

Rule: choose the wrapper that matches your protocol; mixing text and binary streams causes bugs

Socket Options: Tuning TCP Behaviour

- `setSoTimeout(ms)`: `read()` throws `SocketTimeoutException` after `ms` — prevents hanging reads
- `setReuseAddress(true)`: `SO_REUSEADDR` — lets server rebind to same port after crash/restart
- `setTcpNoDelay(true)`: `TCP_NODELAY` — disables Nagle's algorithm; sends small packets immediately
- `setKeepAlive(true)`: `SO_KEEPALIVE` — OS sends keepalive probes to detect dead connections
- `setSendBufferSize(n)` / `setReceiveBufferSize(n)`: tune OS TCP send/receive buffer sizes
- Important: `setReuseAddress()` must be called BEFORE `bind()` (i.e. before `new ServerSocket()`)

Client Socket vs Server Socket

Server Socket

- Listens for incoming connections
- Bound to a specific port
- Passive: waits for clients
- Accepts multiple connections
- Handles multiple clients
- Does not directly send/receive data

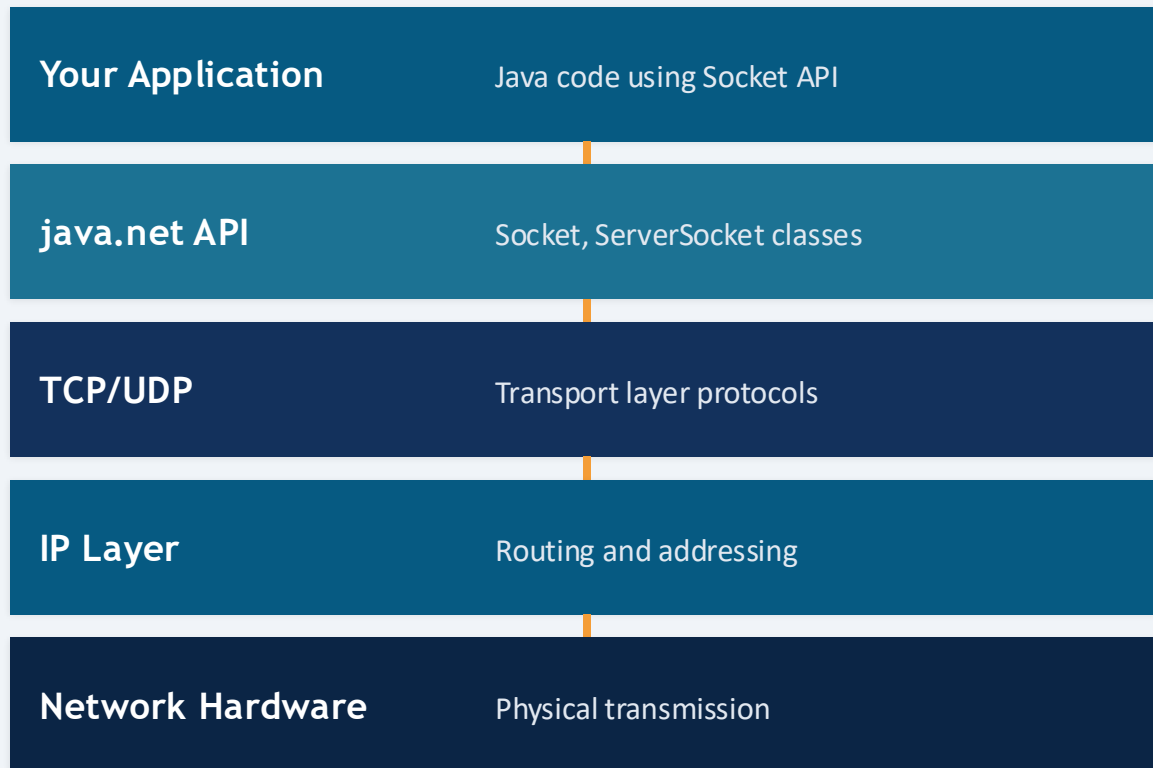
Client Socket

- Initiates connection to server
- Connects to server's port
- Active: initiates communication
- One connection per socket
- Communicates with one server
- Sends and receives data directly

TCP as a Stream: The Boundary Problem

- TCP is a byte stream — it does NOT preserve message boundaries between `send()` calls
- Two writes: `write('Hello') + write('World')` may arrive as `'HelloWorld'` or `'Hell' + 'oWorld'`
- This is the framing problem: how does the receiver know where one message ends?
- Solution 1 (Delimiter): append a special byte like `\n`; receiver reads until that byte
- Solution 2 (Length-Prefix): sender writes a 4-byte int (message length) before the data
- Failure to handle framing = silent data truncation or merging bugs in production

Socket Programming Stack



PART 2

Designing TCP Protocols

Framing, text & binary protocols

Message Framing: Two Strategies

Delimiter-Based (e.g. \n)

- Append a special character (\n) at end of each message
- Receiver reads until delimiter: `readLine()`
- Simple to implement and debug (human-readable)
- Drawback: delimiter must NOT appear in message data
- Fix: escape the delimiter, or use a different byte
- Best for: text commands, chat protocols, HTTP headers

Length-Prefix (Binary)

- Sender writes 4-byte integer (message length) before payload
- Receiver reads 4 bytes first, then reads exactly that many bytes
- Works with any binary data — no special characters needed
- Use `DataOutputStream.writeInt(len) + write(data)`
- Use `DataInputStream.readInt() + readFully(buf)`
- Best for: binary protocols, file transfer, high-throughput

Text Protocol: BufferedReader + PrintWriter

```
// Server-side: text protocol with newline framing
Socket socket = ss.accept();
BufferedReader in = new BufferedReader(
    new InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(
    new OutputStreamWriter(socket.getOutputStream()), true); // auto-flush

String request = in.readLine(); // blocks until \n received
if ("HELLO".equals(request)) {
    out.println("WELCOME"); // println() appends \n and auto-flushes
} else if ("PING".equals(request)) {
    out.println("PONG");
} else {
    out.println("ERROR: unknown command");
}
```

true in PrintWriter = auto-flush on println(). readLine() strips the \n delimiter.

Binary Protocol: DataInputStream + DataOutputStream

```
// Binary protocol: 4-byte length-prefix framing
DataOutputStream out = new DataOutputStream(
    new BufferedOutputStream(socket.getOutputStream()));
DataInputStream in = new DataInputStream(
    new BufferedInputStream(socket.getInputStream()));

// Send: write length, then payload
byte[] data = "PING".getBytes(StandardCharsets.UTF_8);
out.writeInt(data.length);    // 4-byte length prefix
out.write(data);              // actual payload
out.flush();                  // must flush manually (no auto-flush!)

// Receive: read length, then read EXACTLY that many bytes
int len = in.readInt();       // reads exactly 4 bytes
byte[] buf = new byte[len];
in.readFully(buf);            // blocks until all len bytes received
```

readFully() is critical — it retries until all bytes arrive. Plain read() may return fewer bytes.

Real World: HTTP/1.1 is Just Text Over TCP

REAL WORLD CONNECTION

What your code looks like:

```
// fetch() / HttpClient / OkHttp
// all hide this from you:

HttpRequest req = HttpRequest.newBuilder()
    .uri(URI.create(
        "http://example.com/api"))
    .GET().build();

HttpResponse<String> r =
    client.send(req, ...);
```

What travels over the TCP socket:

```
GET /api HTTP/1.1\r\n
Host: example.com\r\n
User-Agent: Java/17\r\n
Connection: close\r\n
\r\n

# Server replies:
HTTP/1.1 200 OK\r\n
Content-Length: 42\r\n
\r\n
{"status":"ok","data":"hello"}
```

Try it live: `nc example.com 80` then type: `GET / HTTP/1.1\r\nHost: example.com\r\n\r\n`

Designing an Application Protocol

- Message format: text (human-readable, debuggable) vs binary (compact, fast)
- Framing strategy: decide delimiter vs length-prefix before writing any code
- Command set: define what operations are supported (GET, SET, DELETE, SUBSCRIBE...)
- Request/response pairing: every command needs a defined success and error response
- Versioning: include a protocol version field — protocols always evolve
- Example: HTTP is text + `\r\n\r\n` delimiter; SMTP uses line-based commands; Redis uses RESP binary

Protocol State Machines

- A protocol connection is a state machine: each message transitions to a new state
- Example FTP states: CONNECTING -> AUTHENTICATING -> IDLE -> TRANSFERRING -> CLOSED
- Track state per connection: each ClientThread holds its own state variable (enum)
- Reject invalid state transitions: cannot transfer before authenticating
- Design tip: draw the state diagram first — code follows naturally from the diagram
- Java: use enum for states — clear, type-safe, and works great with switch expressions

PART 3

Building Multi-Client Servers

Thread pools, state management & broadcasting

Why Do We Need a Multithreaded Server?

- **Problem with single-threaded server:**
 - `accept()` → handle client → `close` → `accept()` next client
 - Client 2 WAITS while Client 1 is being served — terrible for interactive applications
- **Real impact:**
 - If Client 1 takes 5 seconds, Client 2 waits 5 seconds before even connecting
 - For 10 clients queued up, Client 10 might wait 50 seconds — unacceptable
- Solution: give each client its own thread so handling is parallel
- **Thread-per-client model:**
 - Main thread loops on `accept()` only
 - Each accepted socket is handed off to a new `ClientThread` immediately
 - Main thread returns to `accept()` in microseconds — always ready for the next connection
- Result: hundreds of clients can be served simultaneously, each in their own thread

Real-Life Analogy: The Restaurant

Single-Threaded Server

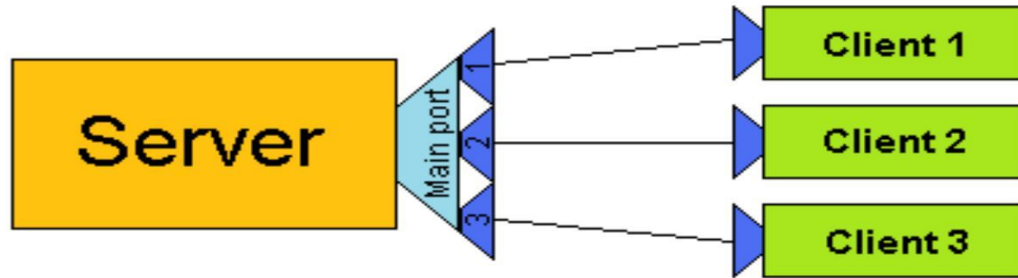
- ONE waiter handles ALL tables
- Waiter takes order from Table 1...
- ...goes to kitchen, waits for food...
- ...serves Table 1...
- THEN finally goes to Table 2
- Table 2 waited through ALL of Table 1's meal
- Result: unhappy customers, poor scalability

Multithreaded Server

- Maitre d' (main thread) seats each guest
- Each table gets its OWN dedicated waiter (thread)
- Waiters work in parallel — no waiting for each other
- Maitre d' is always free to seat the NEXT guest
- Kitchen (CPU) serves all waiters concurrently
- Result: excellent throughput and responsiveness

Multithreaded Server: System View

An illustration of Multithreaded system



Main thread listens on one port; each new client gets a dedicated socket and thread

The accept() Method: Deep Dive

- **The call:**
 - `Socket clientSocket = serverSocket.accept();`
- **What happens:**
 - Thread blocks here — it suspends, consuming zero CPU
 - OS queues incoming TCP SYN requests in the backlog (up to backlog limit)
 - When a client completes the 3-way handshake, `accept()` unblocks and returns
 - The returned `Socket` is already fully connected — handshake is complete
- **The pattern — immediately hand off and loop:**
 - `while (running) { Socket s = ss.accept(); new Thread(new ClientHandler(s)).start(); }`
- **What NOT to do:**
 - Do NOT do heavy work between `accept()` and spawning a thread
 - Do NOT let exceptions in one client crash the `accept()` loop — wrap in `try/catch`
- **Thread naming tip:**
 - `Thread t = new Thread(handler); t.setName("Client-" + clientId++); t.start();` — helps with debugging
- The `ClientThread` receives the `Socket` and owns it — reads requests, writes responses, then closes the socket

Protocol Table: Array Sum Example

Step	Client	Server
1	Connects to server	Accepts connection
2	Sends array size n (int)	Receives n
3	Sends n integers (one by one)	Receives n integers
4	Waits for response	Calculates sum of the n integers
5	Receives sum (int)	Sends sum to client
6	Prints result, closes socket	Closes connection socket

Code Example: Simple TCP Echo Server

```
public class EchoServer {  
    public static void main(String[] args) {  
        try {  
            ServerSocket ss = new ServerSocket(5555);  
            System.out.println("Listening on 5555");  
  
            while (true) {  
                Socket socket = ss.accept();  
                InputStream in = socket.getInputStream();  
                OutputStream out = socket.getOutputStream();  
  
                byte[] data = new byte[256];  
                int bytes = in.read(data);  
                out.write(data, 0, bytes); // Echo back  
  
                socket.close();  
            }  
        } catch (IOException e) { e.printStackTrace(); }  
    }  
}
```

Simple blocking server - handles one client at a time

Code Example: TCP Client

```
public class EchoClient {  
    public static void main(String[] args) {  
        try {  
            Socket socket = new Socket("localhost", 5555);  
            InputStream in = socket.getInputStream();  
            OutputStream out = socket.getOutputStream();  
  
            String msg = "Hello, server!";  
            out.write(msg.getBytes());  
  
            byte[] data = new byte[256];  
            int bytes = in.read(data);  
            System.out.println("Echo: " +  
                new String(data, 0, bytes));  
  
            socket.close();  
        } catch (IOException e) { e.printStackTrace(); }  
    }  
}
```

Client connects, sends message, reads response

Thread Pool Server with ExecutorService

```
public class PooledServer {  
    public static void main(String[] args) throws Exception {  
        ServerSocket ss = new ServerSocket(5555);  
        // Fixed pool: limits concurrent threads, reuses them  
        ExecutorService pool = Executors.newFixedThreadPool(10);  
  
        while (true) {  
            final Socket client = ss.accept();  
            pool.execute(() -> {    // submit task – runs in a pooled thread  
                try {  
                    handleClient(client);  
                } catch (IOException e) {  
                    e.printStackTrace();  
                }  
            });  
        }  
    }  
}
```

Thread pool prevents resource exhaustion. 10,000 clients with thread-per-client = OutOfMemoryError.

Server State: Tracking Connected Clients

```
// Thread-safe set of all active client writers
Set<PrintWriter> clients =
    Collections.synchronizedSet(new HashSet<>());

// In ClientThread.run() – register on connect:
clients.add(out);

// On disconnect (finally block):
clients.remove(out);
socket.close();

// Iterate safely – synchronize on the set:
synchronized (clients) {
    for (PrintWriter pw : clients) {
        pw.println("[Server] " + message);
    }
}
```

Never use plain HashSet from multiple threads — use synchronizedSet() or CopyOnWriteArraySet.

Broadcasting Messages to All Clients

```
// Broadcast helper – called from each ClientThread
public void broadcast(String message, PrintWriter sender) {
    synchronized (clients) {
        for (PrintWriter pw : clients) {
            if (pw != sender) {        // don't echo back to sender
                pw.println(message);
            }
        }
    }
}

// In the client's read loop:
String line;
while ((line = in.readLine()) != null) {
    broadcast "[" + username + "]: " + line, out);
}
```

synchronized block is required — iterating the set while other threads add/remove causes ConcurrentModificationException.

Code: Multithreaded Server Main Loop

```
public class ChatServer {  
    public static void main(String[] args) {  
        try {  
            ServerSocket ss = new ServerSocket(5555);  
            System.out.println("Server listening on 5555");  
  
            int clientCount = 0;  
            while (true) {  
                Socket socket = ss.accept();  
                clientCount++;  
                System.out.println("Client #" + clientCount);  
  
                ClientThread ct = new ClientThread(socket);  
                ct.start();  
            }  
        } catch (IOException e) { e.printStackTrace(); }  
    }  
}
```

Each client is handled in its own thread, non-blocking

Code: Client Thread Implementation

```
public class ClientThread extends Thread {
    Socket socket;

    public ClientThread(Socket s) { socket = s; }

    public void run() {
        try {
            InputStream in = socket.getInputStream();
            OutputStream out = socket.getOutputStream();

            String msg = new String(in.readAllBytes());
            String reply = "Echo: " + msg;
            out.write(reply.getBytes());

            socket.close();
        } catch (IOException e) { e.printStackTrace(); }
    }
}
```

Each thread independently handles one client

PART 4

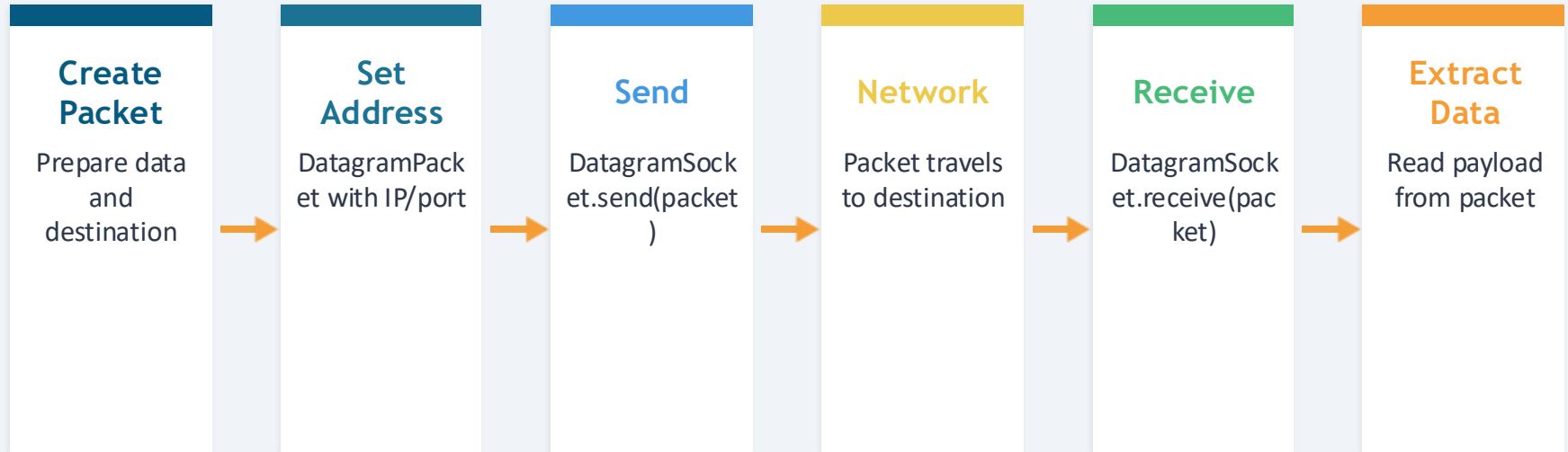
UDP Programming Deep Dive

DatagramSocket API, reliability & multicast

UDP: DatagramSocket API In Depth

- `DatagramSocket()`: creates a UDP endpoint — no connection is established
- `DatagramPacket(data, len, address, port)`: packet with destination; used for `send()`
- `DatagramPacket(buf, buf.length)`: packet with buffer only; used as `receive()` container
- `socket.send(packet)`: fires the datagram — no guarantee of delivery or order
- `socket.receive(packet)`: blocks until a datagram arrives; fills packet with data + sender info
- `packet.getAddress()` / `packet.getPort()`: identify the sender of a received datagram

UDP Packet Journey



Adding Reliability to UDP: Stop-and-Wait ACK

```
// Simple stop-and-wait: send, wait for ACK, retry on timeout
DatagramSocket socket = new DatagramSocket();
int seq = 0;

for (int attempt = 1; attempt <= 3; attempt++) {
    String payload = "seq=" + seq + "|data=Hello";
    DatagramPacket pkt = new DatagramPacket(
        payload.getBytes(), payload.length(), address, port);
    socket.send(pkt);
    socket.setSoTimeout(500);    // wait 500ms for ACK
    try {
        DatagramPacket ack = new DatagramPacket(new byte[32], 32);
        socket.receive(ack);    // wait for ACK
        String msg = new String(ack.getData()).trim();
        if ("ACK=" + seq).equals(msg)) break; // success
    } catch (SocketTimeoutException e) { /* retry */ }
}
```

This is stop-and-wait (one packet at a time). Production: use sliding window. TCP does this automatically.

Code Example: UDP Server

```
public class UDPServer {  
    public static void main(String[] args) {  
        try {  
            DatagramSocket socket = new DatagramSocket(5555);  
            byte[] buffer = new byte[256];  
  
            while (true) {  
                DatagramPacket packet = new DatagramPacket(  
                    buffer, buffer.length);  
                socket.receive(packet);  
                String msg = new String(packet.getData());  
                System.out.println("Received: " + msg);  
                // Send response  
                byte[] response = "Got it!".getBytes();  
                DatagramPacket reply = new DatagramPacket(  
                    response, response.length,  
                    packet.getAddress(), packet.getPort());  
                socket.send(reply);  
            }  
        } catch (IOException e) { e.printStackTrace(); }  
    }  
}
```

Connectionless - can receive from any client

Code Example: UDP Client

```
public class UDPClient {  
    public static void main(String[] args) {  
        try {  
            DatagramSocket socket = new DatagramSocket();  
  
            String msg = "Hello UDP!";  
            byte[] data = msg.getBytes();  
            InetAddress address = InetAddress  
                .getByName("localhost");  
  
            DatagramPacket packet = new DatagramPacket(  
                data, data.length, address, 5555);  
            socket.send(packet);  
  
            byte[] response = new byte[256];  
            DatagramPacket reply = new DatagramPacket(  
                response, response.length);  
            socket.receive(reply);  
            System.out.println(new String(reply.getData()));  
  
            socket.close();  
        } catch (IOException e) { e.printStackTrace(); }  
    }  
}
```

No connection needed - just send packet

UDP Multicast: One-to-Many Communication

- Multicast: one sender delivers to many receivers — without individual unicast copies
- Uses Class D IP addresses: 224.0.0.0 to 239.255.255.255 (reserved multicast range)
- Receivers call `joinGroup(address)` to subscribe; `leaveGroup()` to unsubscribe
- TTL (Time To Live): controls how many network hops the multicast packet may traverse
- Use cases: service discovery (mDNS), live video streaming, game lobbies, stock tickers
- Code:

```
MulticastSocket ms = new MulticastSocket(4446);  
ms.joinGroup(InetAddress.getBy_name("230.0.0.1"));
```

PART 5

Socket Robustness & Diagnostics

Exceptions, retries, diagnostics & production patterns

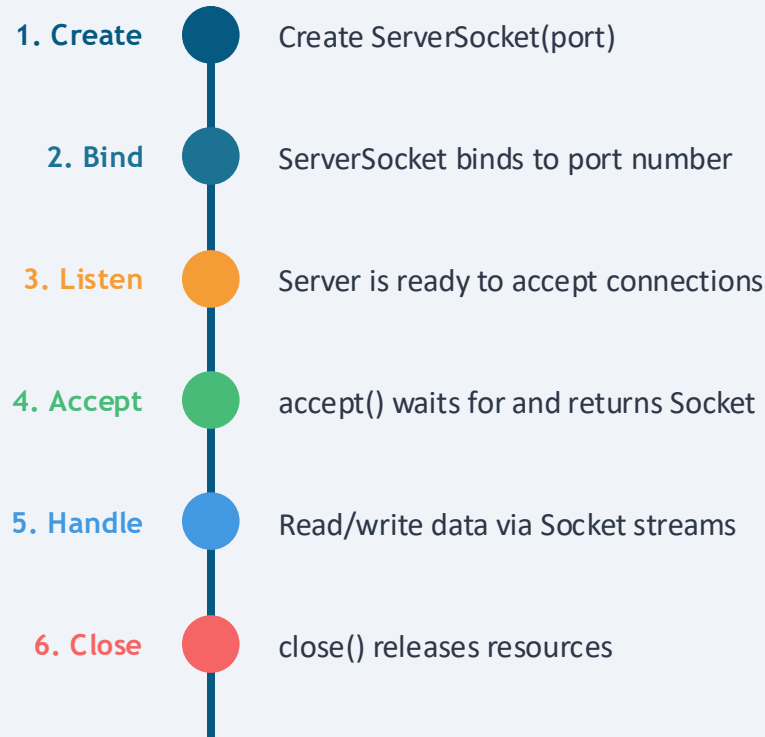
Common Socket Errors and Handling

- `IOException`: General I/O error (read/write/connect)
- `SocketTimeoutException`: Socket timeout during operation
- `ConnectException`: Cannot connect to server (not running?)
- `BindException`: Port already in use (restart needed)
- `SocketException`: Connection reset by peer (unexpected close)
- Must handle with try-catch, proper cleanup in finally/close()

Diagnosing Sockets: Tools of the Trade

- `netstat -an | grep LISTEN`: list all ports your process is bound to
- `ss -tnp` (Linux): modern netstat replacement — shows process name + PID per connection
- `nc (netcat) hostname port`: connect to any TCP server as a raw client — great for quick tests
- `echo -e 'HELLO\n' | nc localhost 5555`: send one line to your server from the command line
- `tcpdump -i lo -A 'tcp port 5555'`: capture and print raw packets on loopback interface
- Wireshark: GUI packet analyser — select 'Follow TCP Stream' to decode your protocol messages

Building a TCP Server: Step by Step



Reconnection with Exponential Backoff

```
public Socket connectWithRetry(String host, int port)
    throws InterruptedException {
    int delay = 1000;    // start: 1 second
    for (int attempt = 1; attempt <= 5; attempt++) {
        try {
            Socket socket = new Socket();
            socket.connect(new InetSocketAddress(host, port), 3000);
            System.out.println("Connected on attempt " + attempt);
            return socket;
        } catch (IOException e) {
            System.err.println("Attempt " + attempt + " failed");
            Thread.sleep(delay);
            delay = Math.min(delay * 2, 30_000);    // max 30s
        }
    }
    throw new RuntimeException("Failed after 5 attempts");
}
```

Exponential backoff prevents thundering herd: all clients retrying simultaneously when server recovers.

PART 6

Real-World Patterns

Half-close, heartbeats, NIO & production patterns

Heartbeat Pattern: Detecting Dead Connections

```
// Problem: TCP may not detect a dead peer for hours (OS default = 2 hours)
// Solution: application-level heartbeat – send PING every 30 seconds
```

```
ScheduledExecutorService scheduler =
    Executors.newScheduledThreadPool(1);

scheduler.scheduleAtFixedRate(() -> {
    List<ClientConnection> dead = new ArrayList<>();
    for (ClientConnection conn : clients) {
        if (!conn.sendPing()) {        // returns false if write fails
            dead.add(conn);
        }
    }
    dead.forEach(ClientConnection::close);
    clients.removeAll(dead);
}, 30, 30, TimeUnit.SECONDS);        // every 30 seconds
```

Combine with `setSoTimeout()` so reads don't block forever between heartbeats.

Non-Blocking I/O: SocketChannel & Selector

- `java.nio.channels.SocketChannel`: NIO alternative to `Socket` — supports non-blocking mode
- `Selector`: monitors multiple channels with ONE thread — the event loop pattern
- `register(selector, SelectionKey.OP_READ)`: tell selector which events to watch per channel
- `selector.select()`: blocks until at least one channel has data — processes a batch of events
- Blocking I/O: thread-per-client fails at 10K+ connections; NIO handles millions per thread
- When to use NIO: build high-throughput servers (APIs, proxies, game servers); use Netty or Vert.x in production

Socket Programming: Patterns at a Glance

- Request-Response: client sends, server replies — HTTP, custom command servers
- Streaming: server pushes data continuously — stock feeds, sensor telemetry, video
- Publish-Subscribe: clients subscribe to topics, server broadcasts — chat, game lobbies
- Pipeline: client sends multiple requests without waiting for each reply — HTTP/2, Redis
- Heartbeat: both sides periodically confirm liveness — avoids zombie connections
- Design tip: pick the pattern that matches your use case before writing a single line of code

Key Takeaways

- 1 TCP is a byte stream — always implement message framing (delimiter or length-prefix)
- 2 Socket options (`SO_TIMEOUT`, `TCP_NODELAY`) tune low-level connection behaviour
- 3 Use thread pools (`ExecutorService`) — never spawn a raw thread per client in production
- 4 Broadcasting requires thread-safe client collections (`synchronizedSet` or `CopyOnWriteArraySet`)
- 5 UDP is connectionless — add reliability (ACKs) explicitly only when needed
- 6 Combine half-close, heartbeats & retry strategies for robust production socket code

Bug Diagnosis #4: The Slow-Motion Crash

BUG HUNT

Think & discuss: 2 minutes

Broken code:

```
ServerSocket server = new ServerSocket(8080);
while (true) {
    Socket s = server.accept();
    new Thread(() -> {
        handleClient(s);
        // s is NEVER closed!
    }).start();
}
// After ~1000 clients: crash
```

Symptom observed:

- Server starts fine, handles clients for hours
- Gradually slows, then crashes
- java.io.IOException: Too many open files
- OS still shows sockets in TIME_WAIT state (lsof -i :8080)
- Restarting server fails: 'Address already in use'

Fix: Wrap with try-with-resources: `try (Socket s = server.accept()) { handleClient(s); }` — Java calls `s.close()` automatically.
TIME_WAIT is normal: OS holds port ~60s after close.

Questions?

Next: Lecture 5 — Multithreaded Servers & Serialization

CSCI 2450 | Computer Interface Programming | 2026